

README - Q1: Matrix Squaring

Description

In this project, we have implemented matrix multiplication by itself using both CPU and GPU. Our main objective was to compare their performance and understand how changing the number of CUDA threads affects execution time.

We tested two matrix sizes of :

- 1000 × 1000
- 2000 × 2000

Implemented Methods

We used three different methods for this project:

- **CPU MATRIX MULTIPLICATION:** Performs matrix multiplication sequentially, one operation at a time.
- **BASIC MATRIX MULTIPLICATION IN PARALLEL :** Uses multiple GPU threads to perform matrix multiplication in parallel.
- **TILED MATRIX MULTIPLICATION:** Uses shared memory to reduce repeated memory access and improve performance.

For both CUDA methods, we tested different block sizes to see how they affect execution time. The grid size is calculated automatically based on the matrix size and block size.

Performance Measurement

We have measured the execution time of each method by running every configuration five times and calculating the averaged we used the **gettimeofday()** function for time measurement.

We have calculated speedup using the following formula:

Speedup = Average CPU Time / Average CUDA Time

Result Verification

We have also checked whether the CPU and GPU produced matching results. For this, we used a tolerance of (0.01)

- **PASS:** The GPU result is within the allowed tolerance of the CPU result.
- **FAIL:** The GPU result differs from the CPU result by more than the allowed tolerance.

Saving the Results

After completing the experiments, the program saves the results in a CSV file named `q1_results.csv`.

The file contains:

- Matrix size
- Implementation used
- Block size
- Threads per block
- Grid dimensions
- Run number
- Execution time
- Average execution time
- Speedup
- Correctness status

We have used these results to prepare the performance comparison and graphs.

GNUplot

After collecting the execution times and speedup values, we used GNUplot to visualize and compare the performance of the different implementations.

We prepared data from the CSV results and used GNUplot scripts to generate graphs

Output

After running the program, the results are saved in a CSV file. This file contains the execution times and other performance details, which can be used to create the graphs in Excel and for the GNUplot.

Executing the Program

First, compile the program using: `nvcc q1_matrix_square.cu -o q1`

Then, run the executable: `./q1`

The program requires an NVIDIA GPU / we can use t4 gpu google collab runtime set to t4 gpu and a working CUDA environment.

Files

- `q1_matrix_square.cu` – Contains the code for the matrix multiplications of above three
- `q1_results.csv` – Contains the execution times, average times, speedup, and correctness results generated by the program.
- `q1_summary.csv` -Contains average of all times